

References I

The formal bibliography (author/year entries resolved from `manuscript/references.bib` via the `[@citekey]` citations used throughout this manuscript) is generated by `pandoc-crossref/natbib` and appears immediately below this note. Every entry was verified this session via a live fetch (Crossref API, DBLP, or the publisher's own DOI resolver) — not taken from training-data memory — before being added to `references.bib`; the fetch evidence is recorded in `ISA.md` `## Verification`.

Annotated pointers, for readers who want the “why this reference” context that a bare bibliography entry doesn't carry:

References II

- ▶ **Maturana & Varela (1980), *Autopoiesis and Cognition*** [[@maturana_varela_1980](#)] — the source of this project's own name. The book defines autopoiesis as the self-producing organization of living systems: a network of processes that continuously regenerates the components that in turn realize the network. This exemplar borrows the word deliberately, not decoratively — `expand()` and `materialize()` are the “self-producing” processes, and `grammar.py`'s seed is the invariant that the network regenerates around every run.
- ▶ **Claessen & Hughes (2000), QuickCheck** [[@claessen2000quickcheck](#)] and **MacIver et al. (2019), Hypothesis** [[@maciver2019hypothesis](#)] — the property-based-testing lineage this project's own test suite descends from. `tests/test_property_invariants.py` uses Hypothesis directly (not merely an homage) to check invariants like “expansion is deterministic for any seed” across generated inputs rather than hand-picked examples.

References III

- ▶ **Lamb & Zacchioli (2022), Reproducible Builds**
[@reproducible_builds] — the peer-reviewed framing for what manuscript/05_reproducibility.md claims operationally: that a build is reproducible when independent re-execution from the same source and environment produces bit-for-bit-identical (or hash-identical) output, and that this is a supply-chain-integrity property, not merely a convenience.
- ▶ **Merkle (1987), digital signatures / hash trees**
[@merkle_tree_provenance] — the theoretical basis for src/integrity.py's content-addressed provenance: a tree of hashes lets a verifier recompute and confirm the integrity of a large structure from its leaves up, without trusting the producer's say-so. This project's integrity_profile:merkle grammar slot is a direct, if much smaller-scale, application of that idea.